LAB 7

Fuzz Testing for Input Validation Bugs

Daycare Fee Processor | Lab 6 application continued

Name	B Varun	**SRN**	PES1UG24CS109
Section	B	**Date**	11 November 2026

Objective and result

The Lab 6 daycare fee application was fuzz-tested for malformed inputs. The valid-input business rules remain the same. The three functions now validate types and ranges before calculations, and return clear TypeError or ValueError messages for invalid inputs.

Issues found in the original app

Input	Original behavior	Correction
Sibling flag = 'false'	Truthy string silently changed the fee	Require an actual bool
Age = True	Boolean was accepted as an integer	Reject bool for integer counts
Minutes = 19.5	Fractional minutes produced a status	Require integer minutes
Hours = '31'	Unclear low-level TypeError	Raise a clear hours validation error

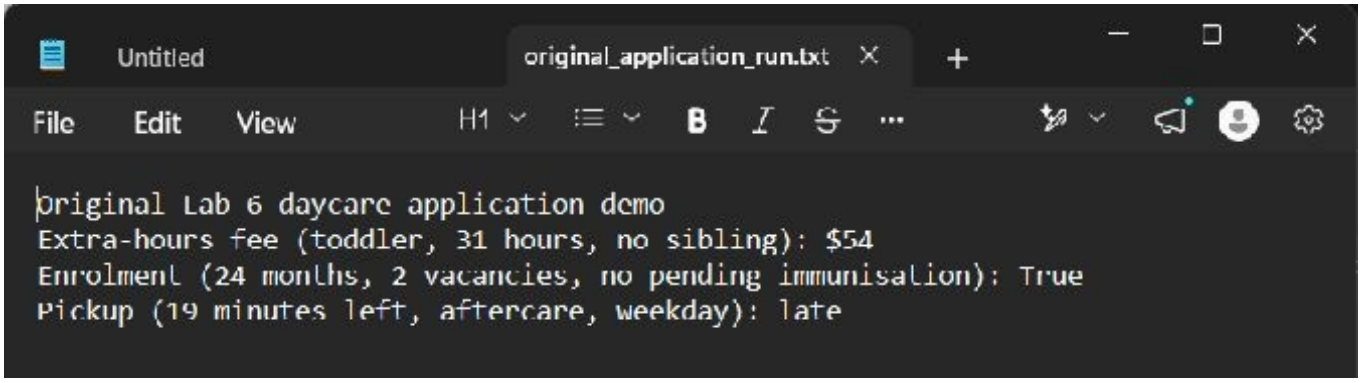
Final verification

37 tests passed - pytest 9.2.0 and Hypothesis 6.170.15. The suite contains regression examples and deterministic generated cases for extra-hours fees, enrolment, and pickup status.

Evidence note: screenshots show the captured application and pytest output in a clean Notepad window; the exact stdout is also preserved as text files in the evidence folder.

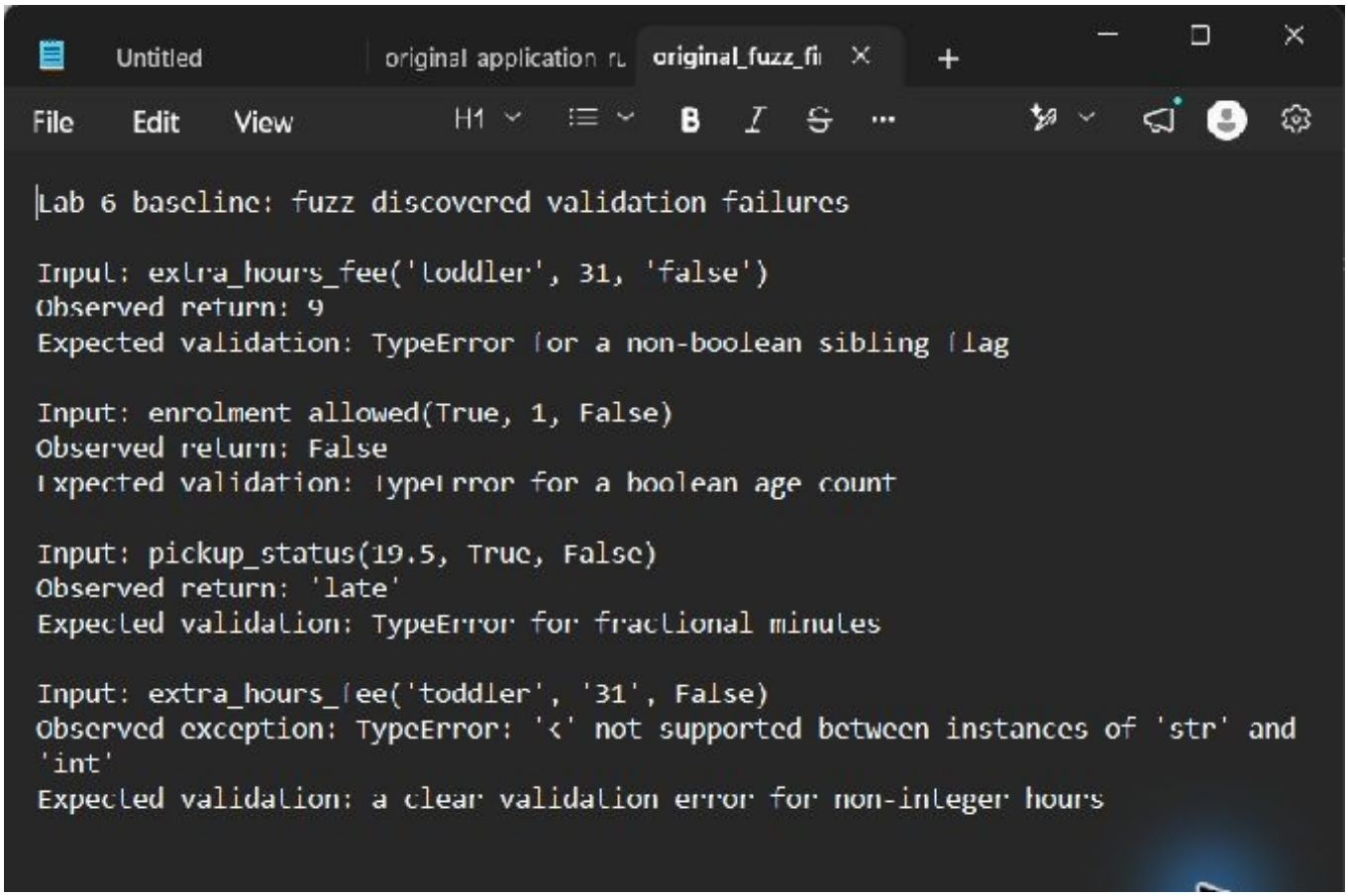
Execution evidence: original application

Valid examples from the unmodified Lab 6 daycare module.



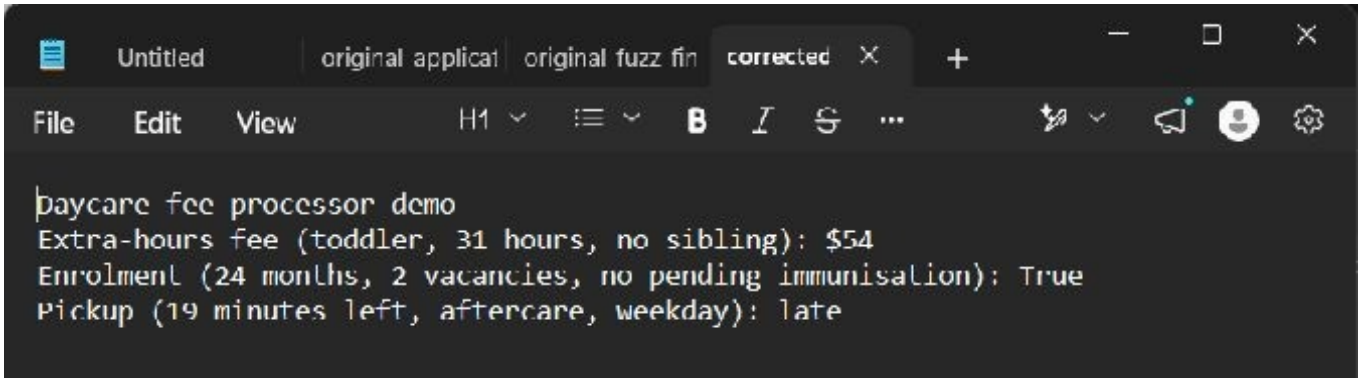
Fuzz findings from the original application

Malformed input examples show where the original code silently accepted bad types or raised an unclear low-level exception.



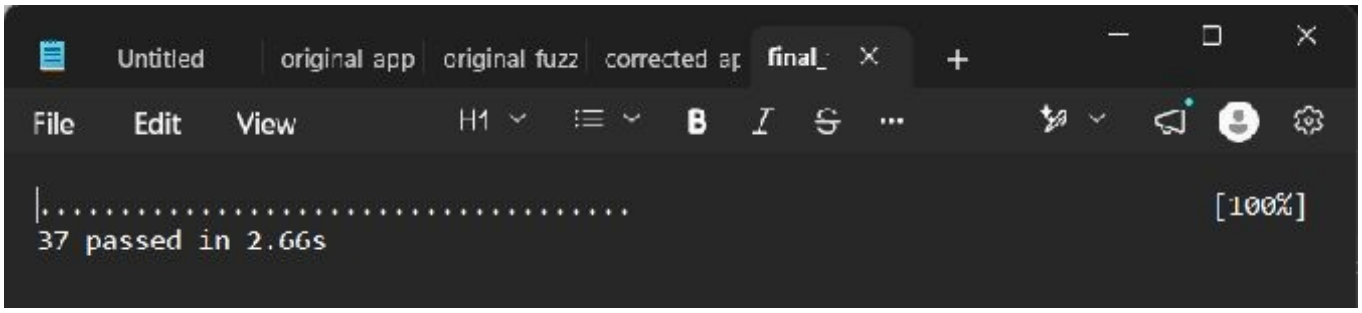
Execution evidence: corrected application

The fixed processor preserves the same valid-example output.



Final fuzz-test execution

The final run completed successfully: 37 regression and Hypothesis fuzz tests passed.



Command: `python -B -m pytest -p no:cacheprovider -q`

Source code: processor.py

Validated implementation of the three Lab 6 application functions.

```
"""Validated daycare-fee processor used by Lab 7."""

INCLUDED_HOURS = {"infant": 20, "toddler": 25, "preschool": 30}


def _require_bool(value, name):
    if type(value) is not bool:
        raise TypeError(f"{name} must be a boolean")


def _require_integer(value, name, *, minimum=None):
    # bool is an int subclass, but it is never a valid count of hours/months.
    if type(value) is not int:
        raise TypeError(f"{name} must be an integer")
    if minimum is not None and value < minimum:
        raise ValueError(f"invalid {name}")


def extra_hours_fee(age_group, hours, has_sibling):
    """Return the fee for care beyond the included hours for an age group."""
    if not isinstance(age_group, str):
        raise TypeError("age_group must be a string")
    if age_group not in INCLUDED_HOURS:
        raise ValueError("unknown age group")
    _require_integer(hours, "hours", minimum=0)
    _require_bool(has_sibling, "has_sibling")

    included = INCLUDED_HOURS[age_group]
    if has_sibling and age_group != "infant":
        included += 5
    extra = max(0, hours - included)
    if extra == 0:
        return 0

    rate = 14 if age_group == "infant" else 9
    if extra > 20:
        rate += 5
        if age_group == "infant":
            rate += 3
    return extra * rate


def enrolment_allowed(age_months, vacancies, is_immunisation_pending):
    """Return whether a child meets the age and vacancy enrolment rules."""
    _require_integer(age_months, "age_months", minimum=0)
    _require_integer(vacancies, "vacancies", minimum=0)
    _require_bool(is_immunisation_pending, "is_immunisation_pending")

    if is_immunisation_pending:
        return age_months >= 12
    if age_months < 6:
        return False
    return vacancies >= 1


def pickup_status(minutes_left, is_aftercare, is_friday):
    """Classify pickup timing using the weekday and aftercare cutoffs."""
    _require_integer(minutes_left, "minutes_left")
    _require_bool(is_aftercare, "is_aftercare")
    _require_bool(is_friday, "is_friday")

    cutoff = 20 if is_aftercare else 10
    if is_friday:
        cutoff += 10
    if minutes_left >= cutoff:
        return "on_time"
    if minutes_left >= cutoff - 15:
        return "late"
    return "overdue"


def main():
    """Run a small, repeatable demonstration of the three app functions."""
    print("Daycare fee processor demo")
    fee = extra_hours_fee("toddler", 31, False)
    enrolled = enrolment_allowed(24, 2, False)
    status = pickup_status(19, True, False)
    print(f"Extra-hours fee (toddler, 31 hours, no sibling): ${fee}")
    print(f"Enrolment (24 months, 2 vacancies, no pending immunisation): {enrolled}")
    print(f"Pickup (19 minutes left, aftercare, weekday): {status}")


if __name__ == "__main__":
    main()
```

Test source: test_processor.py

Regression cases and Hypothesis-based fuzz tests for all three functions.

```
"""Regression and Hypothesis fuzz tests for all three processor functions."""

import pytest
from hypothesis import given, settings, strategies as st

from processor import extra_hours_fee, enrolment_allowed, pickup_status


FUZZ = settings(max_examples=80, deadline=None, derandomize=True)
AGE_GROUPS = ("infant", "toddler", "preschool")


@pytest.mark.parametrize(
    "age_group,hours,has_sibling,expected",
    [
        ("infant", 20, False, 0),
        ("infant", 21, False, 14),
        ("toddler", 25, False, 0),
        ("toddler", 31, False, 54),
        ("preschool", 35, True, 0),
        ("toddler", 51, True, 294),
        ("infant", 42, True, 484),
    ],
)
def test_extra_hours_fee_regressions(age_group, hours, has_sibling, expected):
    assert extra_hours_fee(age_group, hours, has_sibling) == expected


@FUZZ
@given(
    age_group=st.sampled_from(AGE_GROUPS),
    hours=st.integers(min_value=0, max_value=250),
    has_sibling=st.booleans(),
)
def test_fuzz_extra_hours_fee_matches_fee_rules(age_group, hours, has_sibling):
    included = {"infant": 20, "toddler": 25, "preschool": 30}[age_group]
    if has_sibling and age_group != "infant":
        included += 5
    extra = max(0, hours - included)
    rate = 14 if age_group == "infant" else 9
    if extra > 20:
        rate += 5 + (3 if age_group == "infant" else 0)
    assert extra_hours_fee(age_group, hours, has_sibling) == extra * rate


@FUZZ
@given(age_group=st.text().filter(lambda value: value not in AGE_GROUPS))
def test_fuzz_extra_hours_fee_rejects_unknown_age_groups(age_group):
    with pytest.raises(ValueError, match="unknown age group"):
        extra_hours_fee(age_group, 0, False)


@FUZZ
@given(age_group=st.one_of(st.none(), st.integers(), st.lists(st.text())))
def test_fuzz_extra_hours_fee_rejects_non_string_age_groups(age_group):
    with pytest.raises(TypeError, match="age_group must be a string"):
        extra_hours_fee(age_group, 0, False)


@FUZZ
@given(hours=st.one_of(st.floats(), st.text(), st.none(), st.booleans(), st.lists(st.integers())))
def test_fuzz_extra_hours_fee_rejects_non_integer_hours(hours):
    with pytest.raises(TypeError, match="hours must be an integer"):
        extra_hours_fee("toddler", hours, False)


@FUZZ
@given(hours=st.integers(max_value=-1))
def test_fuzz_extra_hours_fee_rejects_negative_hours(hours):
    with pytest.raises(ValueError, match="invalid hours"):
        extra_hours_fee("toddler", hours, False)


@FUZZ
@given(has_sibling=st.one_of(st.integers(), st.text(), st.none(), st.lists(st.integers())))
def test_fuzz_extra_hours_fee_rejects_non_boolean_sibling_flag(has_sibling):
    with pytest.raises(TypeError, match="has_sibling must be a boolean"):
        extra_hours_fee("toddler", 31, has_sibling)


@pytest.mark.parametrize(
    "age_months,vacancies,pending,expected",
    [
        (12, 0, True, True),
```

```
        (11, 5, True, False),
        (5, 5, False, False),
        (6, 1, False, True),
        (6, 0, False, False),
    ],
)
def test_enrolment_regressions(age_months, vacancies, pending, expected):
    assert enrolment_allowed(age_months, vacancies, pending) is expected


@FUZZ
@given(
    age_months=st.integers(min_value=0, max_value=180),
    vacancies=st.integers(min_value=0, max_value=50),
    pending=st.booleans(),
)
def test_fuzz_enrolment_allowed_matches_enrolment_rules(age_months, vacancies, pending):
    expected = age_months >= 12 if pending else age_months >= 6 and vacancies >= 1
    assert enrolment_allowed(age_months, vacancies, pending) is expected


@FUZZ
@given(age_months=st.integers(max_value=-1))
def test_fuzz_enrolment_rejects_negative_age(age_months):
    with pytest.raises(ValueError, match="invalid age_months"):
        enrolment_allowed(age_months, 1, False)


@FUZZ
@given(vacancies=st.integers(max_value=-1))
def test_fuzz_enrolment_rejects_negative_vacancies(vacancies):
    with pytest.raises(ValueError, match="invalid vacancies"):
        enrolment_allowed(12, vacancies, False)


@FUZZ
@given(
    value=st.one_of(
        st.floats(), st.text(), st.none(), st.booleans(), st.lists(st.integers())
    )
)
def test_fuzz_enrolment_rejects_non_integer_counts(value):
    with pytest.raises(TypeError):
        enrolment_allowed(value, 1, False)
    with pytest.raises(TypeError):
        enrolment_allowed(12, value, False)


@FUZZ
@given(pending=st.one_of(st.integers(), st.text(), st.none(), st.lists(st.integers())))
def test_fuzz_enrolment_rejects_non_boolean_pending_flag(pending):
    with pytest.raises(TypeError, match="is_immunisation_pending must be a boolean"):
        enrolment_allowed(12, 1, pending)


@pytest.mark.parametrize(
    "minutes_left,is_aftercare,is_friday,expected",
    [
        (20, True, False, "on_time"),
        (19, True, False, "late"),
        (4, True, False, "overdue"),
        (10, False, False, "on_time"),
        (9, False, False, "late"),
        (-6, False, False, "overdue"),
        (30, True, True, "on_time"),
        (20, True, True, "late"),
        (14, True, True, "overdue"),
        (20, False, True, "on_time"),
        (10, False, True, "late"),
    ],
)
def test_pickup_status_regressions(minutes_left, is_aftercare, is_friday, expected):
    assert pickup_status(minutes_left, is_aftercare, is_friday) == expected


@FUZZ
@given(
    minutes_left=st.integers(min_value=-500, max_value=500),
    is_aftercare=st.booleans(),
    is_friday=st.booleans(),
)
def test_fuzz_pickup_status_matches_cutoff_rules(minutes_left, is_aftercare, is_friday):
    cutoff = (20 if is_aftercare else 10) + (10 if is_friday else 0)
    expected = "on_time" if minutes_left >= cutoff else (
        "late" if minutes_left >= cutoff - 15 else "overdue"
    )
    assert pickup_status(minutes_left, is_aftercare, is_friday) == expected


@FUZZ
```

```
@given(minutes_left=st.one_of(st.floats(), st.text(), st.none(), st.booleans(), st.lists(st.integers())))
def test_fuzz_pickup_status_rejects_non_integer_minutes(minutes_left):
    with pytest.raises(TypeError, match="minutes_left must be an integer"):
        pickup_status(minutes_left, False, False)


@FUZZ
@given(flag=st.one_of(st.integers(), st.text(), st.none(), st.lists(st.integers())))
def test_fuzz_pickup_status_rejects_non_boolean_flags(flag):
    with pytest.raises(TypeError):
        pickup_status(15, flag, False)
    with pytest.raises(TypeError):
        pickup_status(15, False, flag)
```